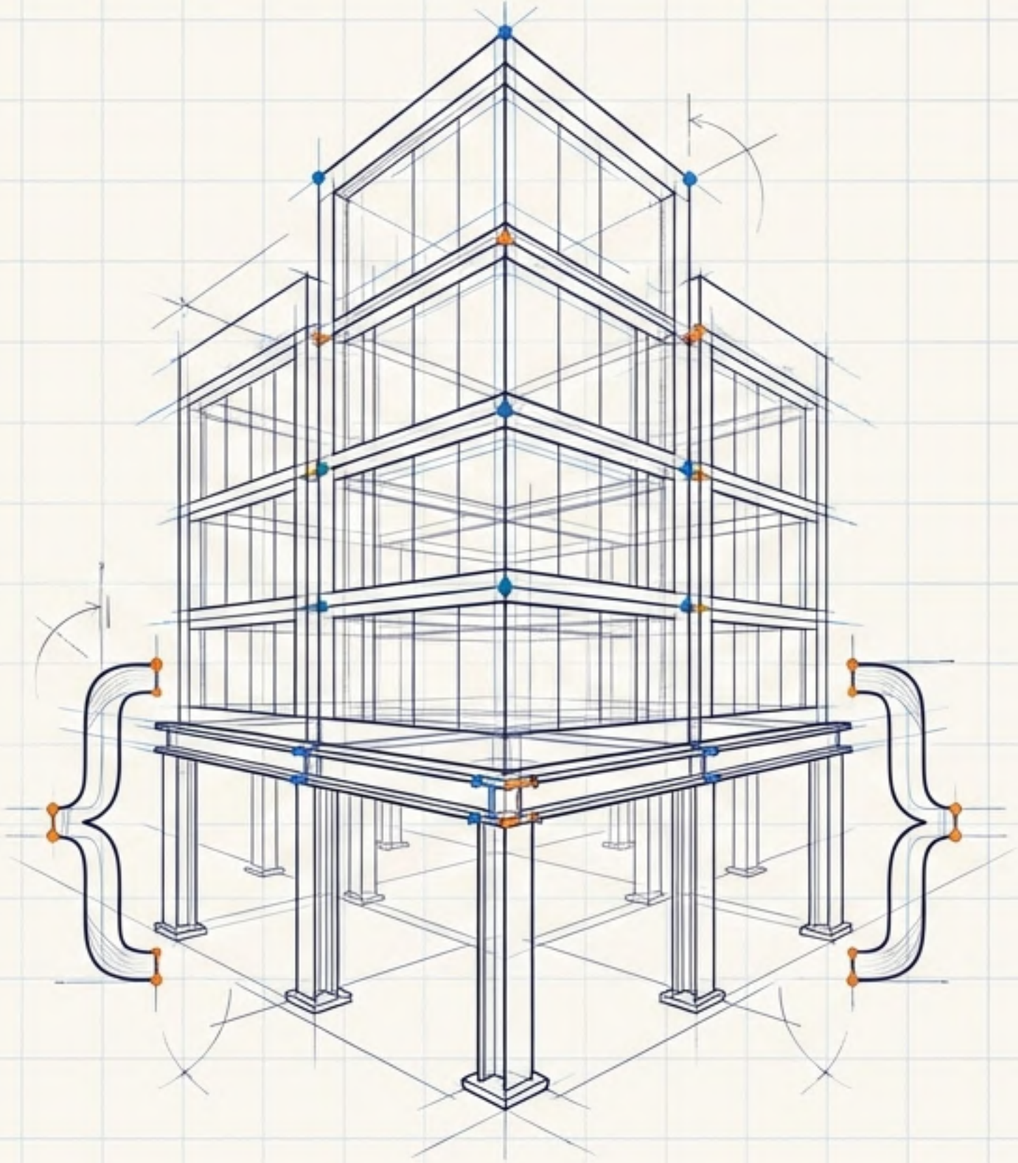
Sass Moderno: Arquitectura Escalable y Módulos

Un manual visual para refactorizar código heredado.

De variables globales a encapsulamiento estricto.



NotebookLM

El Problema del Código Heredado

_buttons.scss

```
$primary-color: blue;
```

_forms.scss

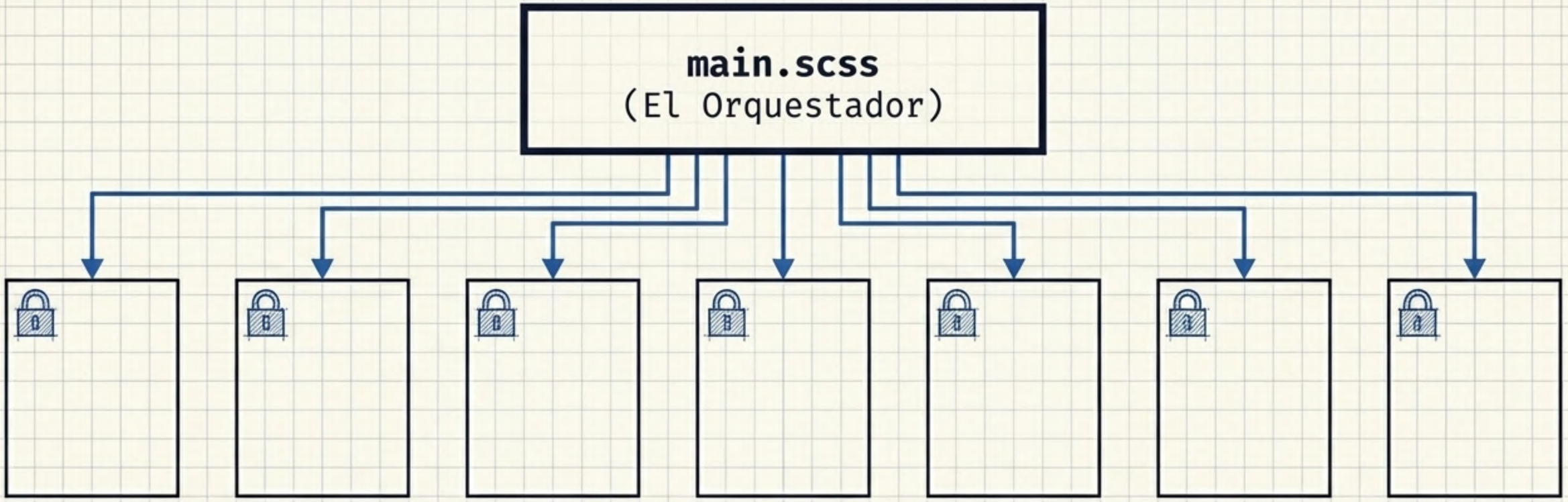
```
$primary-color: red;
```

Colisión de Variables:
El último archivo importado sobrescribe silenciosamente al anterior, causando errores de interfaz impredecibles.

La directiva tradicional **@import** coloca todas las **variables**, **mixins** y **funciones** en un único **entorno global**, generando **dependencias invisibles** y **compilaciones lentas**.

NotebookLM

Los Cimientos de un Sistema Escalable



El Patrón 7-1 es el estándar de la **industria** para organizar bases de código Sass de cualquier tamaño.

Separación estricta de responsabilidades:
7 carpetas de origen,
1 único archivo de salida.

NotebookLM

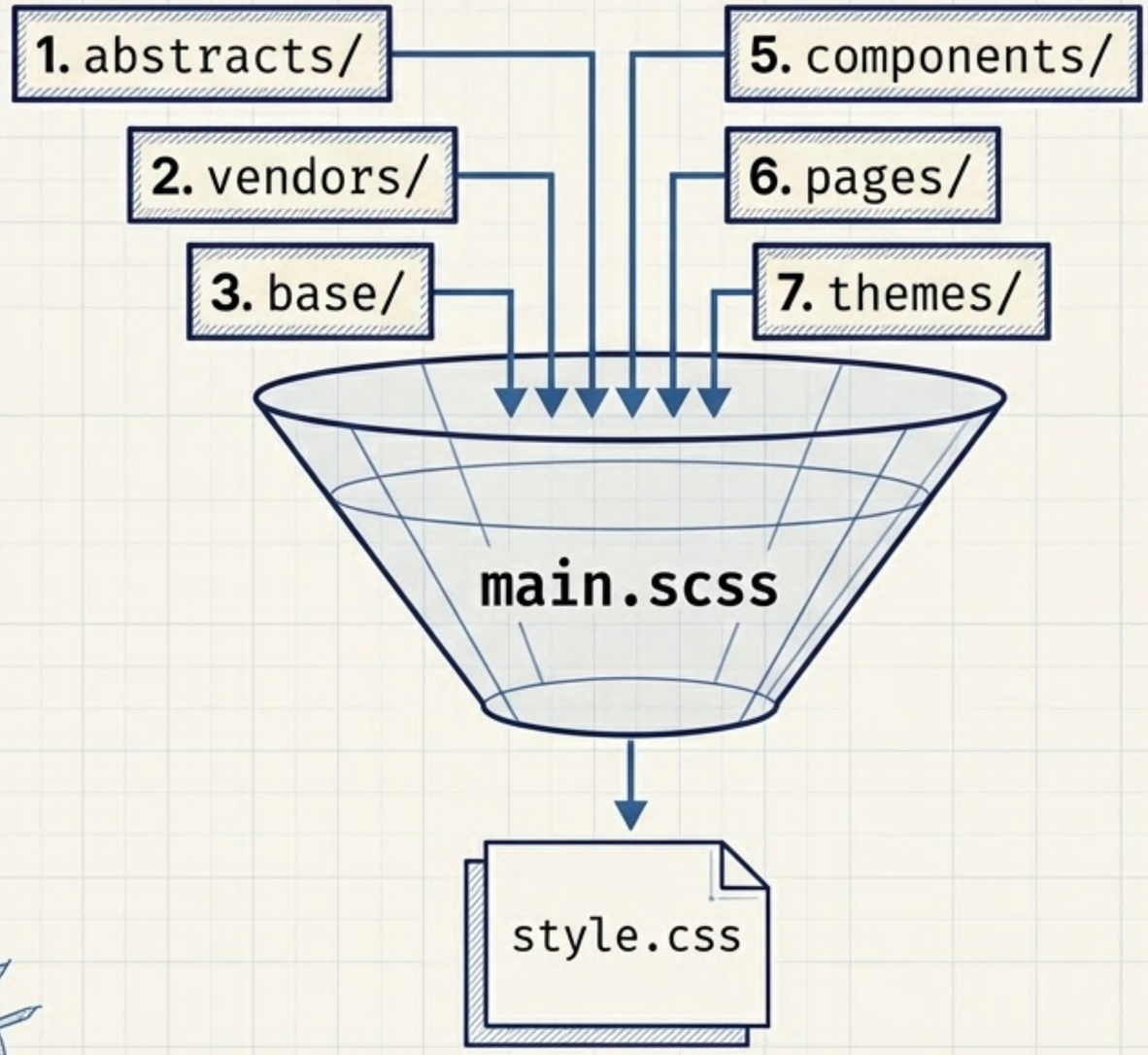
Anatomía del Patrón 7-1

```
🗁 sass/
  ├ 🗀 base/
  ├ 🗀 components/
  ├ 🗀 layout/
  ├ 🗀 pages/
  ├ 🗀 themes/
  ├ 🗀 abstracts/
  └ 🗀 vendors/
```

- `base/` - Reglas fundamentales (Resets, tipografía base).
- **components/** - Micro-estructuras reutilizables (Botones, tarjetas).
- **layout/** - Macro-estructuras (Grid, header, footer).
- **pages/** - Estilos específicos de una vista única.
- **themes/** - Variaciones visuales (Modo oscuro, alto contraste).
- **abstracts/** - Herramientas sin salida CSS (Variables, mixins, funciones).
- **vendors/** - Código de terceros (Bootstrap, Normalize).

NotebookLM

El Orquestador Único



`main.scss` es el único archivo que no comienza con un guion bajo (_).

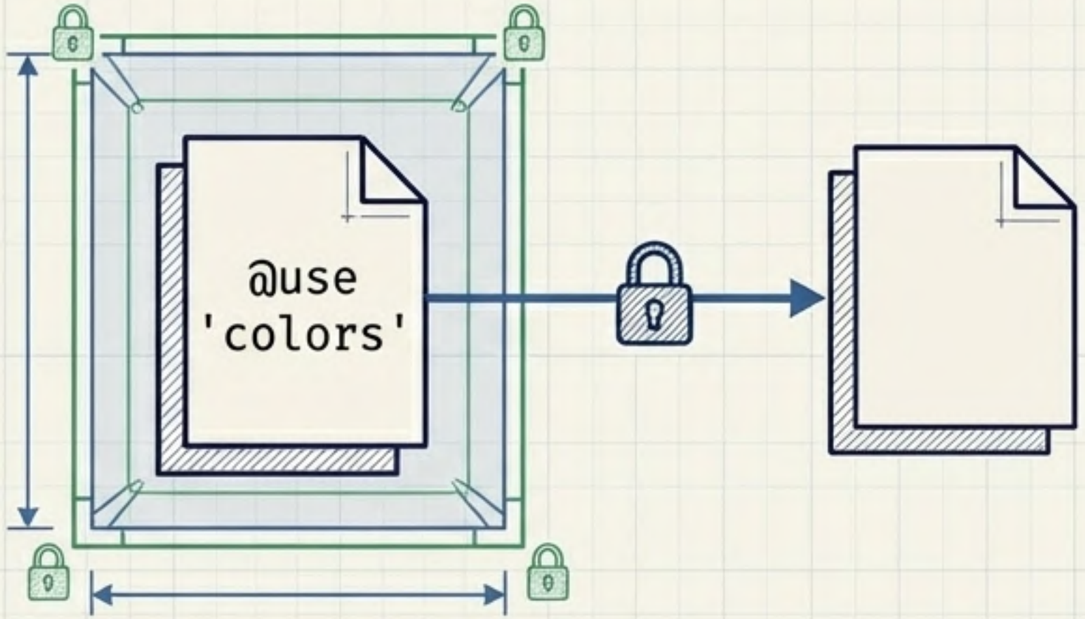
Solo contiene directivas de importación, nunca código CSS directo.

El orden de carga es crítico para la cascada: las herramientas abstractas y dependencias externas (abstracts, vendors) deben cargarse antes de construir los componentes y layouts.

NotebookLM

La Muerte Anunciada de @import

La documentación oficial de Sass declara `@import` obsoleto. Será eliminado del lenguaje.



`@use` carga el archivo como un MÓDULO, aislando su contenido y evitando la contaminación del entorno global.

NotebookLM

Encapsulamiento Mediante Namespaces

_variables.scss

```
$primary: #ffaa00;
```

_button.scss

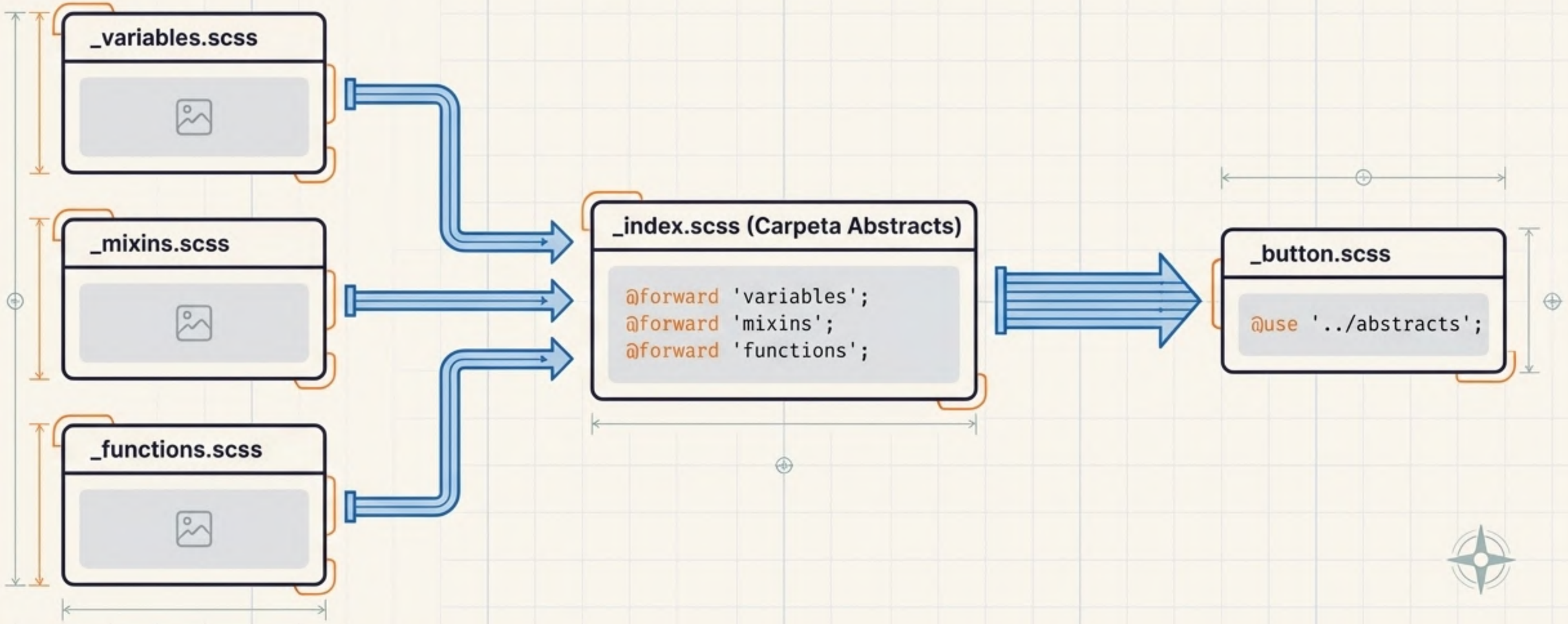
```
@use '../abstracts/variables';

background-color: variables.$primary;
```

- Los módulos cargados con @use requieren un namespace por defecto (el nombre del archivo).
- El código sabe exactamente de dónde proviene cada variable, eliminando la ambigüedad.

NotebookLM

Reenviando Dependencias con @forward



1. `@forward` permite agrupar múltiples módulos y exponerlos bajo una única API limpia.
2. El componente final solo necesita una única declaración `@use` para acceder a todo el sistema de diseño.

NotebookLM

Matriz de Comportamiento del Compilador

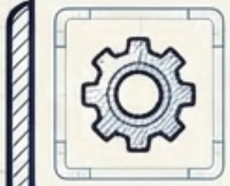
Por qué el nuevo sistema es superior en rendimiento y seguridad.

Característica	**Directiva** `@import`	**Directiva** `@use`
Alcance (Scope)	Global (Contaminante)	Local (Namespace estricto)
Compilación Múltiple	Duplica el CSS si se importa varias veces.	Compila una sola vez, sin importar las llamadas.
Dependencias Internas	Accesibles implícitamente (Magia).	Requiere importación explícita.
Estado Actual	Deprecado ⚠	Estándar Moderno ✓

NotebookLM

La Era de los Módulos Integrados (Built-in)

~~darken() nth() map-get() lighten()~~



sass:math

Cálculos precisos y funciones matemáticas

sass:color

Manipulación y análisis de color avanzado

sass:map

Operaciones con estructuras de datos clave-valor

sass:list

Gestión y manipulación de listas de elementos

Las funciones globales mágicas han desaparecido. Sass ahora requiere importar explícitamente sus motores internos como módulos utilizando **@use** para prevenir colisiones con funciones nativas futuras de CSS.

NotebookLM

Refactorizando Matemáticas: La División

Obsoleto

```
width: $container / 2;
```

El operador / es ahora ambiguo, ya que CSS nativo lo utiliza como separador (ej. Grid, rgb).

Moderno

```
@use 'sass:math';
width: math.div($container, 2);
```

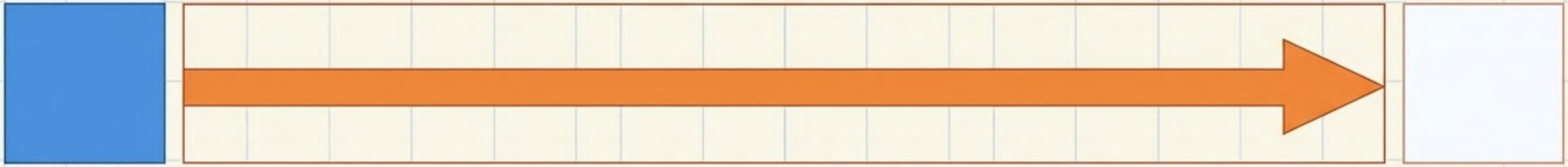
Función explícita y segura para cálculos compilados matemáticamente.

La función explícita `math.div()` asegura que la operación se compile matemáticamente en el preprocesador de forma segura.

NotebookLM

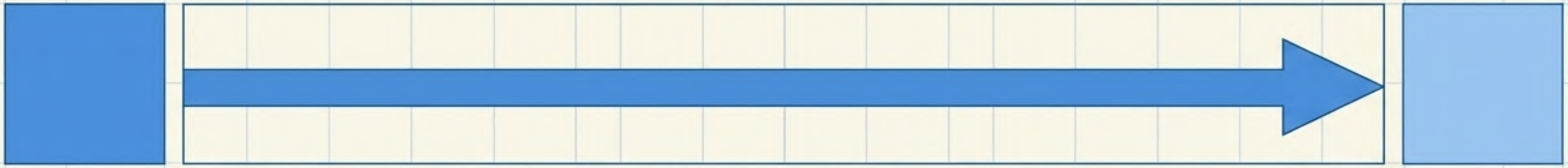
El Problema con la Manipulación de Color

`lighten($color, 20%)`



Interpolación Lineal: Sopla el color al blanco rápidamente, destruyendo la saturación.

`mix($color, white, 20%)`



Mezcla Relativa: Conserva la naturaleza del color original de forma predecible.

Las funciones heredadas `lighten()` y `darken()` alteran el espacio HSL de forma lineal y absoluta, resultando impredecibles en sistemas de diseño modernos.

NotebookLM

Refactorizando Colores

El módulo nativo sass:color proporciona herramientas matemáticas superiores para escalar propiedades de color en lugar de forzarlas.

```
@use 'sass:color';

.btn:hover {
  background-color: color.scale($primary, $lightness: -20%);
}
```

Realiza ajustes fluidos y relativos que respetan los límites del color original.

Alternativa válida para matices puros: `mix(black, $primary, 20%)` crea sombras perfectas sin quemar el color.

NotebookLM

Matriz de Migración Rápida

Código Heredado (Obsoleto)		Código Moderno (Estándar)
`$width / 2`	→	`math.div($width, 2)`
`darken($color, 10%)`	→	`color.scale($color, $lightness: -10%)`
`map-get($theme, primary)`	→	`map.get($theme, 'primary')`
`@import 'buttons'`	→	`@use 'buttons'`
`lighten($color, 15%)`	→	`mix(white, $color, 15%)`

NotebookLM

Síntesis: El Componente Perfecto

components/_button.scss

```
1 | @use 'sass:math';
2 | @use '../abstracts' as a;
3 |
4 | .btn {
5 |   background: a.$brand-primary;
6 |   padding: math.div(a.$spacing-base, 2);
7 |   &:hover { background: mix(black, a.$brand-primary, 10%); }
8 | }
```

Módulos integrados importados explícitamente en la parte superior.

Uso del patrón 7-1 y hub @forward con un namespace personallzado ('a').

Acceso seguro a variables mediante el namespace.

División matemática moderna e inequívoca.

Mezcla de color moderna y predecible.

Un módulo independiente, predecible y conectado a una arquitectura escalable.
El estándar de oro moderno.

NotebookLM